

Builder Pattern (Done by: Anisa Domi, Ledio Tafciu)

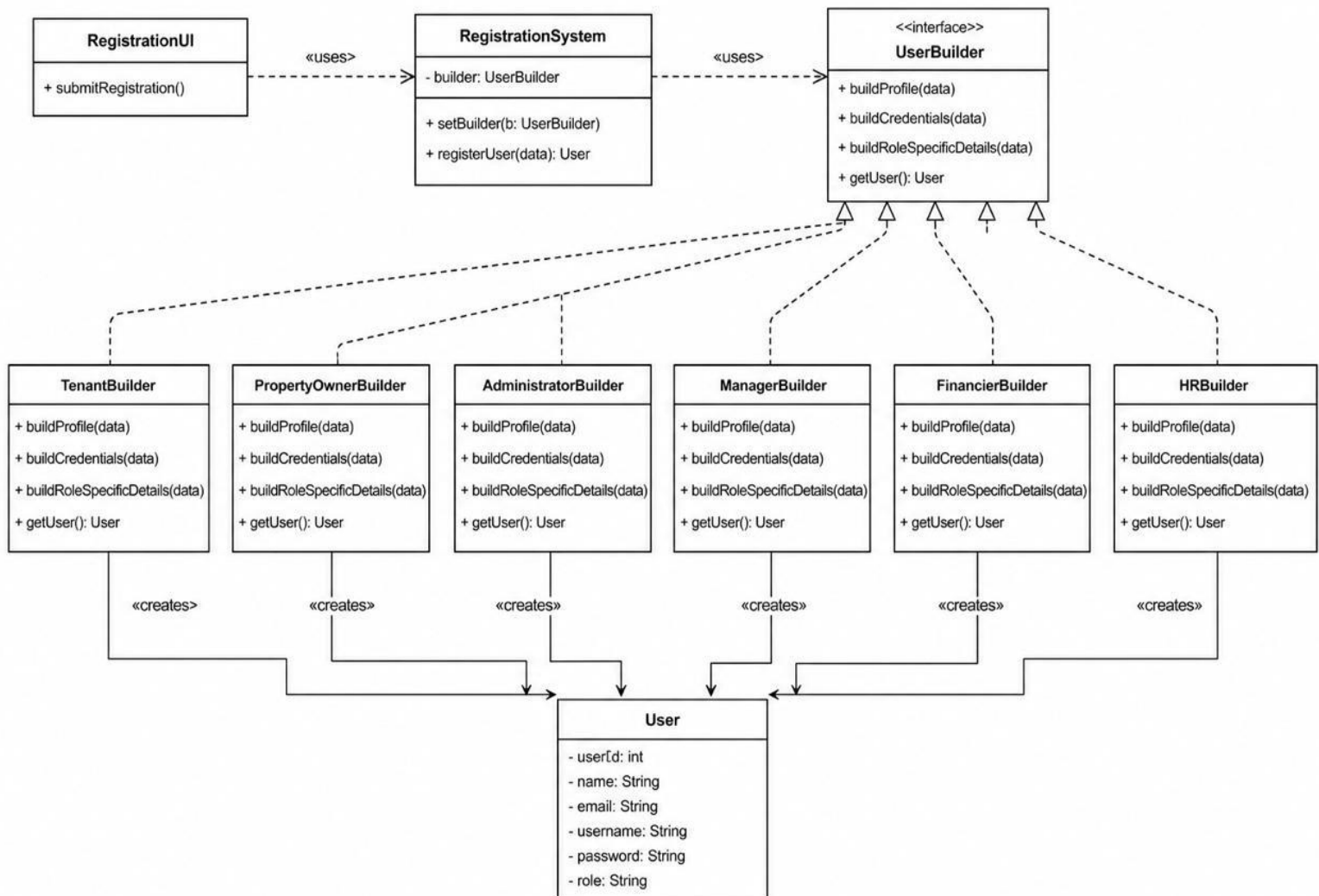
Justifying the design pattern presence → The Builder Pattern is used in the system to automatically construct different User objects during registration based on the selected role.

In this design:

The RegistrationSystem acts as the Director controlling the build sequence. UserBuilder defines the construction steps, each concrete builder assembles role-specific data, and the User product is returned fully built by the system.

For example:

In **FR41** when a user registers an account and system selects the correct builder for the chosen role, builds profile data, credentials, and role-specific details step by step and returns a complete User object without changing the registration flow.



Chain of Responsibility Pattern (Done by:Kevin Muho, Ornelt Lemellari,Matias Bali)

Justifying the design pattern presence → The Chain of Responsibility Pattern is used in the system to automatically process complaints through multiple handling stages without placing all complaint logic in one class.

In this design:

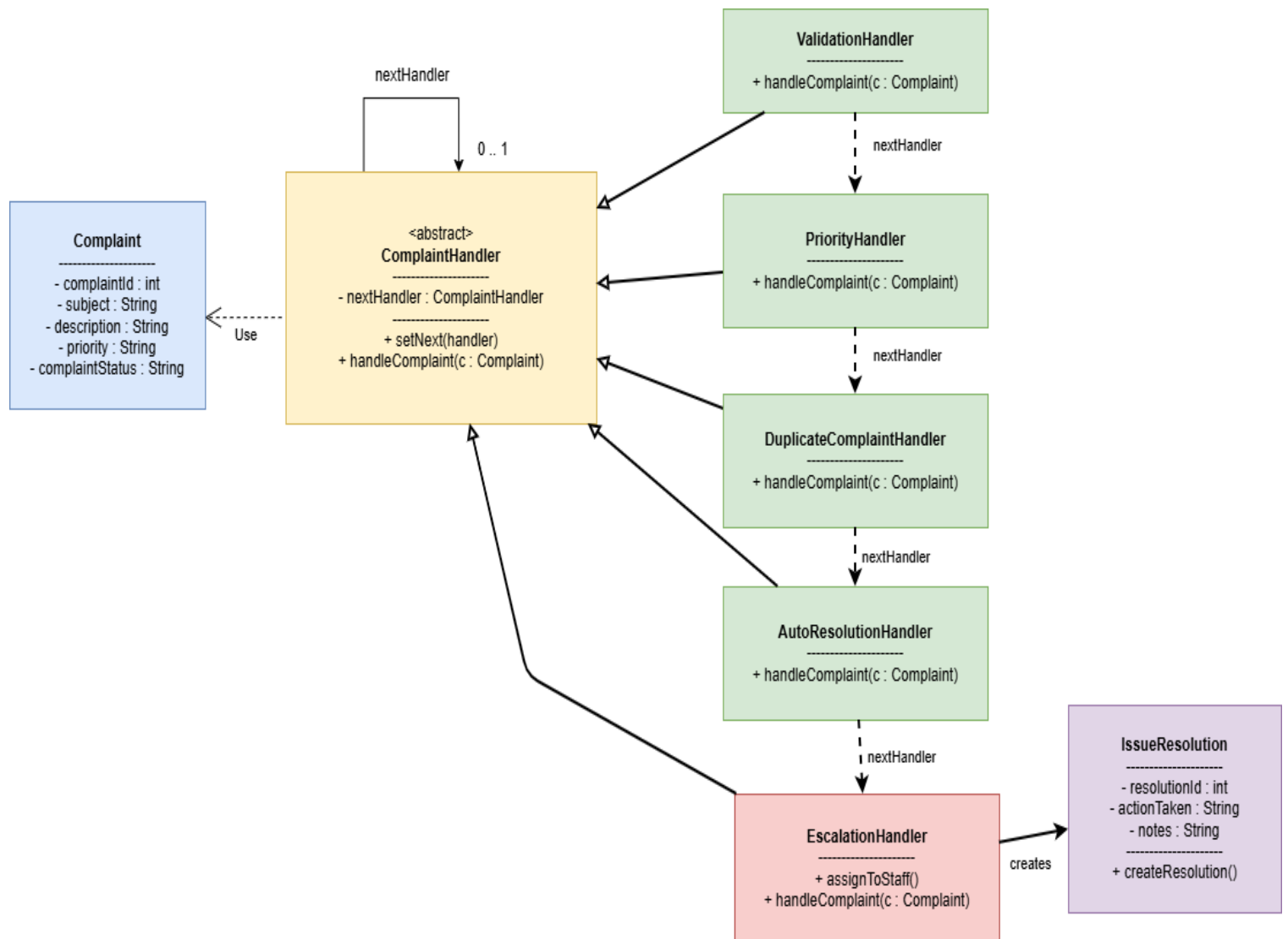
ComplaintHandler acts as the abstract handler and stores nextHandler. ValidationHandler, PriorityHandler, DuplicateComplaintHandler, AutoResolutionHandler, and EscalationHandler act as concrete handlers. Complaint is the request object passed through the chain. IssueResolution stores the final resolution details.

For example:

In **FR38**, when HR manages complaints, the system first validates and categorizes the complaint automatically.

In **FR33**, if the complaint cannot be resolved automatically, it is escalated for manager review.

In **FR26**, unresolved or serious complaints can be escalated to the administrator as a dispute/system issue.



Observer Pattern (Done by:Desard Lybeshari, Haidi Toto)

Justifying the design pattern presence→The Observer Pattern is used in the system to automatically notify users when important events occur, such as rental request updates or payment completion.

In this design:

RentalRequest and Payment act as Subjects because their status can change.

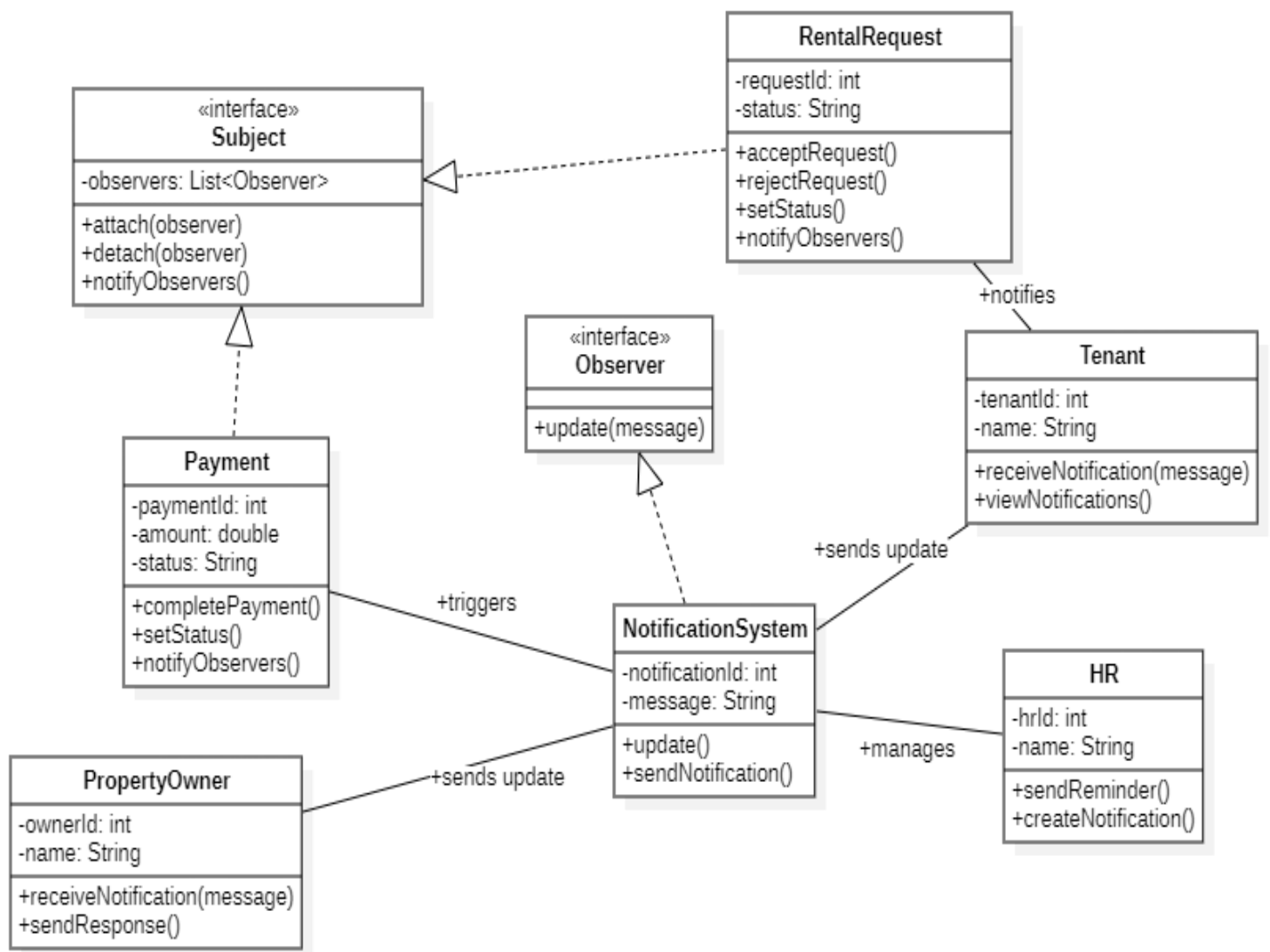
NotificationSystem acts as the Observer, listening for these changes.

Tenant and PropertyOwner receive notifications about approvals, rejections, payments, and reminders.

For example:

-In **FR17**, when a property owner accepts or rejects a rental request, the tenant is notified.

-In **FR8** and **FR39**, payment updates and reminders are also sent automatically.



Facade Pattern (Done by:Kevin Muho, Ornel Lemellari, Matias Bali)

Justifying the design pattern presence→The Facade Pattern is used in the system to simplify the property booking workflow by providing a unified interface for handling multiple booking-related operations through a single facade class.

In this design:

IBookingFacade acts as the facade interface that defines the booking operations.

BookingFacadeImpl acts as the concrete facade implementation that coordinates the subsystem classes.

Property, RentalRequest, Booking, Payment, and Notification act as subsystem classes responsible for specific booking operations.

Tenant acts as the client that interacts with the facade instead of directly communicating with all subsystem classes.

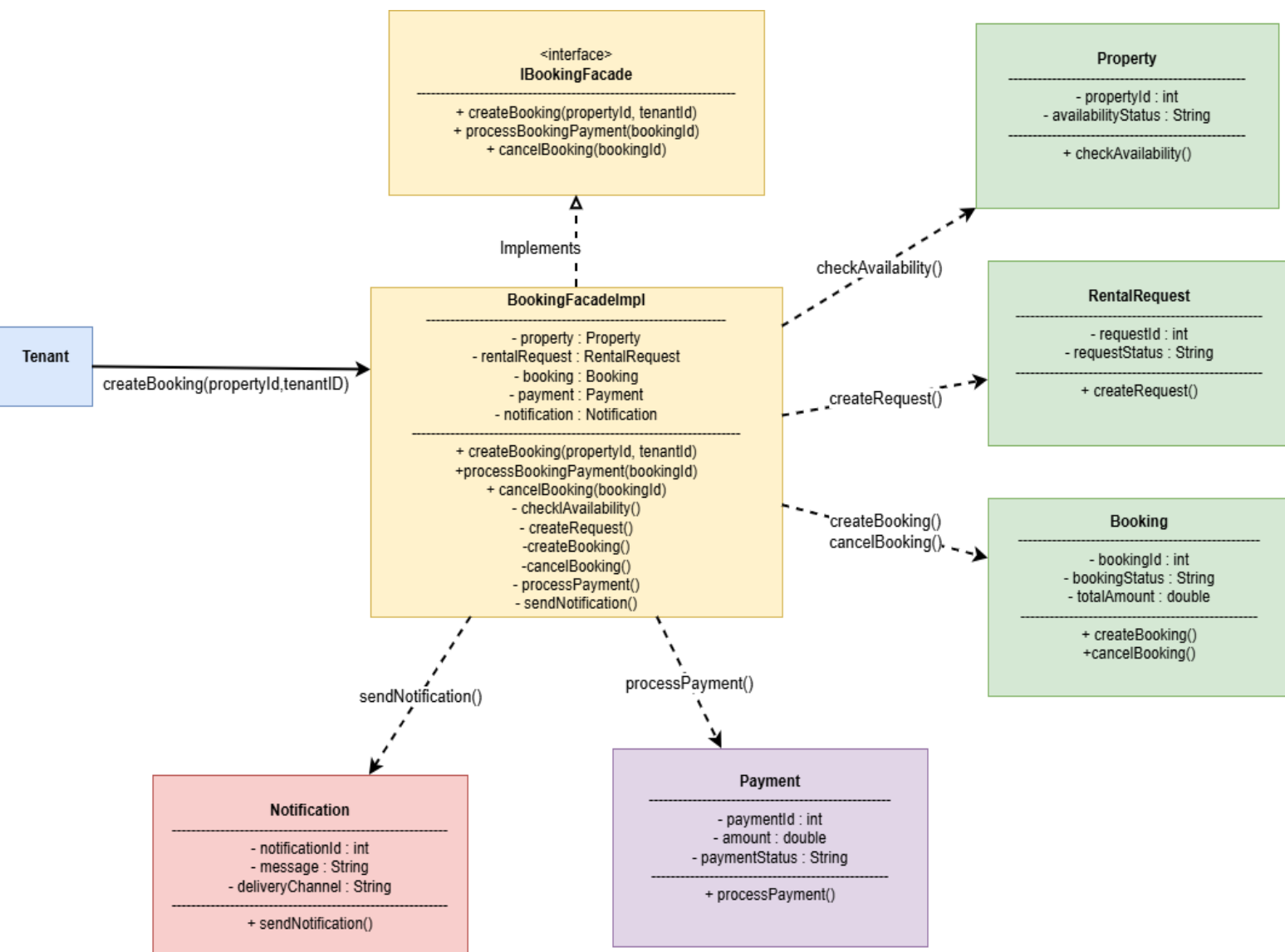
For example:

In **FR2** and **FR3**, the system checks property availability before creating a booking request.

In **FR5**, the system automatically creates a rental request through the facade.

In **FR17**, the system manages booking creation and status updates after request approval.

The facade also sends notifications automatically after important booking and payment events occur.



Adapter Pattern (Done by: Desard Lybeshari, Haidi Toto)

Justifying the design pattern presence → The Adapter Pattern is used to connect the rental system with an external payment service without changing the internal system structure.

In this design:

PaymentServiceAdapter acts as a bridge between the system and ExternalPaymentService. The system sends payment data using the PaymentProcessor interface, while the adapter converts the request and response formats required by the external service.

This pattern focuses on *software structure and object collaboration*, since it shows how objects inside the system communicate during payment processing.

For example:

On **FR8 – Submit Rental Payment**, the system sends the payment request to the payment service. Also, the system receives the payment result and displays confirmation.

